Magpie — Sprint 6 Technical Spec (Phase 4 opener: eBay "Connect")

Audience: implementing agent (Codex) + Regan. **Builds on** Phases 1–3 (`SPRINT_0_1_SPEC.md` … `SPRINT_5_SPEC.md`). **Status: planning / review — implement only after this spec is reviewed and approved.**

> Build only what is in this document (the locked Sprint 6 scope, D-56…D-64). **This sprint connects an eBay account; it does not touch a listing.** No media upload, no inventory item, no offer, no publish — that is Sprint 7. **CI never makes a network call**: fakes only. The first time real eBay traffic happens is the manual sandbox checklist, run by a human, outside CI.

Conventions inherited (not restated): `TimeStampedUUIDModel`; modular monolith under `apps/`; adapter ports under `integrations/` with HTTP confined to the integration module (the D-32 / Sprint 3 metals pattern); DRF defaults; `manage.py backup` must include all new tables; settings split with env-loaded config.

---

0. Locked decisions this spec implements

- **D-56** Phase 4 = two sprints. **Sprint 6 "Connect"** (auth, credentials, account state, audit rails — zero listing writes) → **Sprint 7 "Publish"** (media, inventory item, unpublished offer, human-gated `publishOffer`, sync).
- **D-57** New `apps/ebay`; two focused adapters (`EbayAuthAdapter`, `EbayAccountAdapter`) + fakes + `EbayUnavailable` in `integrations/ebay.py`; `EBAY_ENV=sandbox|production`. No fat adapter; Sprint 7 adds media/inventory adapters later.
- **D-58** `EbayCredential` with **Fernet-encrypted tokens** (env key `MAGPIE_TOKEN_ENCRYPTION_KEY`; connect refuses to run without it); refresh token persisted encrypted; access token disposable, refresh-on-demand, **encrypted if persisted**; `select_for_update` refresh locking; tokens/auth codes never in logs, admin (redacted), API responses, or audit payloads; backups carry ciphertext only.
- **D-59** `AuditLog` lands now (fulfils D-20): thin, append-only, plain target strings, no update/delete API, secret-free payloads, dotted action taxonomy.

- **D-60** Least-privilege scopes requested up-front (F-11): `sell.inventory` + `sell.account.readonly`; escalate to `sell.account` only on verified need; Media/EPS scope is a **build-time verification item** (include in the consent set only if confirmed required for Sprint 7); nothing speculative.
- **D-61** OAuth capture = **paste-back baseline** with full state discipline (pre-generated state, validated, short-expiry, single-use; auth codes never stored; pasted URLs redacted). Localhost callback = sandbox-optional convenience. A public callback endpoint is **not** a Sprint 6 requirement.
- **D-62** CI = fakes only, forever. Sprint 6 closure = green PostgreSQL CI + the manual sandbox checklist (§13) with evidence. Production connect not required to close.
- **D-63** Consolidate CI into `.github/workflows/postgresql-validation.yml` ("PostgreSQL Validation"), running the **full** suite; retire all per-sprint workflows.
- **D-64** Out of scope (§12).

---

1. New apps & repo additions

```
backend/apps/audit/              # NEW — tiny, shared
├── models.py                    # AuditLog
├── services.py                  # record(actor, action, target_type, target_id,
├── serializers.py               # read-only
├── views.py                     # read-only list endpoint
└── tests/

backend/apps/ebay/               # NEW
├── constants.py                 # scopes, audit action names, state TTL
├── fields.py                    # EncryptedTextField (Fernet)
├── models.py                    # EbayCredential, OAuthState, EbayAccountSnapsho
├── services.py                  # connect/capture/refresh/disconnect/policy-refr
├── serializers.py
├── views.py
└── tests/

backend/integrations/ebay.py     # REPLACES the Sprint 0 stub:
                                 # TokenSet, EbayAuthAdapter, EbayAccountAdapter,
                                 # HttpEbayAuthAdapter, HttpEbayAccountAdapter,
                                 # FakeEbayAuthAdapter, FakeEbayAccountAdapter,
                                 # get_ebay_auth_adapter(), get_ebay_account_adap
                                 # EbayUnavailable
```

- `INSTALLED_APPS += "apps.audit", "apps.ebay"` . New migrations expected (four tables).
- **One new third-party dependency:** `cryptography` (Fernet) — the first since Sprint 0; pin it in `requirements.txt` .
- Dependency direction: `ebay` → `audit` ; nothing else imports `ebay` . (Sprint 7's publish flow will be the second `audit` consumer.)
- **HTTP lives only in** `integrations/ebay.py` — never in `apps/` . Token-endpoint request/response bodies are never logged.

---

2. Configuration (env — never committed)

Var	Meaning
`EBAY_ENV`	`sandbox` | `production` (unset ⇒ fakes; app fully functional, status = "not configured")
`EBAY_CLIENT_ID` / `EBAY_CLIENT_SECRET`	The keyset **for the selected env** (operator swaps values when switching env)
`EBAY_RU_NAME`	The RuName (redirect-URL name) for that keyset
`MAGPIE_TOKEN_ENCRYPTION_KEY`	Fernet key (generate via `Fernet.generate_key()`); **connect flow hard-fails without it**; lives outside the DB and outside backups

`constants.py` : `EBAY_SCOPES = ["https://api.ebay.com/oauth/api_scope/sell.inventory", "https://api.ebay.com/oauth/api_scope/sell.account.readonly"]` (+ the Media scope **only if** build-time verification confirms Sprint 7 needs it — D-60); `OAUTH_STATE_TTL_MINUTES = 10` .

---

3. Models

3.1 `ebay.fields.EncryptedTextField`

`TextField` subclass: encrypts with Fernet on write, decrypts on read; key from `MAGPIE_TOKEN_ENCRYPTION_KEY` ; raises a clear `ImproperlyConfigured` if used without the

key. DB stores ciphertext only (test-asserted).

3.2 `ebay.EbayCredential`

```
class EbayCredential(TimeStampedUUIDModel):
    owner = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True, on
    environment = models.CharField(max_length=12)            # sandbox | producti
    ebay_user_id = models.CharField(max_length=120, blank=True, default="")
    ebay_username = models.CharField(max_length=120, blank=True, default="")
    scopes = models.JSONField(default=list, blank=True)       # granted at consen

    refresh_token = EncryptedTextField()                      # ~18 months (F-11)
    refresh_token_expires_at = models.DateTimeField(null=True, blank=True)
    access_token = EncryptedTextField(blank=True, default="") # disposable cache
    access_token_expires_at = models.DateTimeField(null=True, blank=True)

    connected_at = models.DateTimeField(auto_now_add=True)
    last_refresh_at = models.DateTimeField(null=True, blank=True)
    last_refresh_error = models.TextField(blank=True, default="")

    class Meta:
        constraints = [models.UniqueConstraint(fields=["environment"], name="one_
```

Practically one row per environment. **Admin registers it with every token field redacted** (`***`), no change form for token fields. Disconnect = delete the row (audited).

3.3 `ebay.OAuthState`

```
class OAuthState(TimeStampedUUIDModel):
    state = models.CharField(max_length=64, unique=True)     # secrets.token_urls
    expires_at = models.DateTimeField()                       # now + OAUTH_STATE
    consumed_at = models.DateTimeField(null=True, blank=True) # single-use
```

Capture validates: exists, unexpired, unconsumed → marks consumed in the same transaction as the code exchange. Expired/unknown/reused state ⇒ validation error + `ebay.connect.failed` audit entry (state value may appear in the payload; **the auth code never does**).

3.4 `ebay.EbayAccountSnapshot`

```
class EbayAccountSnapshot(TimeStampedUUIDModel):
    environment = models.CharField(max_length=12)
```

```
    business_policies_opted_in = models.BooleanField(null=True)   # null = not ye
    payment_policies = models.JSONField(default=list, blank=True)
    fulfillment_policies = models.JSONField(default=list, blank=True)
    return_policies = models.JSONField(default=list, blank=True)
    fetched_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["environment"], name="one_
```

One row per env, refreshed whole by the policy-refresh service. Own-account data with `fetched_at` provenance (no F-09 concern). This is the readiness panel's source and Sprint 7's policy-selector source.

3.5 `audit.AuditLog` (D-59 / fulfils D-20)

```
class AuditLog(TimeStampedUUIDModel):
    actor = models.CharField(max_length=120, blank=True, default="")   # username
    action = models.CharField(max_length=80)                            # dotted
    target_type = models.CharField(max_length=80, blank=True, default="")
    target_id = models.CharField(max_length=64, blank=True, default="")
    payload = models.JSONField(default=dict, blank=True)                # NEVER t

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["action"]), models.Index(fields=["created
```

Append-only by construction: `audit.services.record(...)` **is the only writer; no update/delete API anywhere**; admin read-only. Sprint 6 actions: `ebay.connect.started / completed / failed`, `ebay.disconnect.completed`, `ebay.token.refresh.completed / failed`, `ebay.policy_refresh.completed / failed`.

---

4. Adapter contracts (`integrations/ebay.py`)

```
@dataclass
class TokenSet:
    access_token: str; access_expires_at: datetime
    refresh_token: str | None; refresh_expires_at: datetime | None

class EbayAuthAdapter(Protocol):
    def build_consent_url(self, *, state: str) -> str: ...
    def exchange_code(self, *, code: str) -> TokenSet: ...          # full TokenS
    def refresh(self, *, refresh_token: str) -> TokenSet: ...       # refresh_tok
```

```
class EbayAccountAdapter(Protocol):
    def get_identity(self, *, access_token: str) -> dict: ...       # {user_id, u
    def get_policy_optin_status(self, *, access_token: str) -> bool: ...
    def list_policies(self, *, access_token: str, kind: str) -> list[dict]: ...
```

- `HttpEbayAuthAdapter` / `HttpEbayAccountAdapter` : real implementations; sandbox vs production base URLs from `EBAY_ENV` ; raise `EbayUnavailable` on network/5xx/config-missing; **never log token bodies**.
- `FakeEbayAuthAdapter` / `FakeEbayAccountAdapter` : deterministic in-memory behaviour for tests + CI (valid/invalid code paths, refresh success/failure toggles, canned identity + policies).
- Factories `get_ebay_auth_adapter()` / `get_ebay_account_adapter()` : env-keyed; **unset `EBAY_ENV` ⇒ fakes** — the app never depends on eBay being configured (D-13).

---

5. Services (`ebay/services.py`)

- **`start_connect()`** → creates `OAuthState` , builds consent URL via the auth adapter, audits `ebay.connect.started` , returns `{consent_url, state}` .
- **`complete_connect(pasted: str)`** → parses `code` + `state` from a pasted full URL *or* accepts the bare pair; validates + consumes the state; exchanges immediately; fetches identity; upserts `EbayCredential` (encrypted); audits `ebay.connect.completed` (payload: env, ebay_username, scopes — **never the code or tokens**). Any failure → `ebay.connect.failed` . The pasted value is never written to any log.
- **`get_access_token()`** (used by account calls; Sprint 7 reuses): returns cached token if >60 s of validity remains; else refresh inside `transaction.atomic()` + `select_for_update()` on the credential row (single-flight); on success update tokens + `last_refresh_at` , audit `ebay.token.refresh.completed` ; on failure record `last_refresh_error` , audit `…failed` , raise `EbayUnavailable` .
- **`disconnect()`** → delete credential row (+ env's snapshot optional keep), audit `ebay.disconnect.completed` . UI confirms first.
- **`refresh_account_snapshot()`** → opt-in status + three policy lists → upsert `EbayAccountSnapshot` with `fetched_at` ; audit completed/failed.

---

6. API endpoints

Method	Path	Purpose
POST	`/api/ebay/connect/start/`	→ `{consent_url}` (state handled server-side).
POST	`/api/ebay/connect/complete/`	Body `{pasted_url}` or `{code, state}`; never logged; → connection summary.
POST	`/api/ebay/disconnect/`	Confirmed in UI; deletes credential.
GET	`/api/ebay/status/`	`{configured, environment, connected, ebay_username, scopes, access/refresh expiries, last_refresh_error, snapshot: {opted_in, policy_counts, fetched_at}}` — **no token values, ever**.
POST	`/api/ebay/refresh-policies/`	Runs `refresh_account_snapshot()`.
GET	`/api/audit-log/`	Read-only, paginated, filterable by `action` prefix + `target_type`. No write methods exist.

---

7. Frontend — Settings → eBay

New small **Settings** route (first one in the app) with an **eBay channel page**: `EbayConnectPanel` (Connect button → shows the consent URL to open in a new tab + a paste field for the redirected URL → Complete), `EbayStatusCard` (connected-as, **prominent SANDBOX / PRODUCTION badge**, scopes, token expiries, last refresh error), `PolicyReadinessCard` (opt-in ✓/✕, counts per policy kind, fetched-at, Refresh button), Disconnect with `ConfirmDialog`, and `AuditLogTable` (read-only latest entries, action filter). New typed API module `ebay.ts` (+ `audit.ts`).

---

8. CI consolidation (D-63)

New `.github/workflows/postgresql-validation.yml`, displayed name **"PostgreSQL Validation"**: Postgres 16 service → migrate → seed → **full backend suite** → frontend tests + typecheck + build. **Delete all per-sprint workflow files** (Sprint 0–5, including

the still-triggering Sprint 4 one). This single workflow is the CI gate for Sprint 6 and every sprint after.

---

9. Acceptance criteria

Done when:

1. Migrations (audit ×1, ebay ×3 tables) apply cleanly on real PostgreSQL in the **consolidated** CI; `manage.py backup` includes all four tables; restore passes; per-sprint workflows are gone.
2. **Encryption proven:** DB-level check that stored token columns are ciphertext (≠ plaintext, Fernet-decryptable round-trip); connect flow raises clearly when `MAGPIE_TOKEN_ENCRYPTION_KEY` is absent; admin shows redacted tokens with no editable token fields.
3. **State discipline proven:** expired, unknown, and reused states are each rejected; the happy path consumes the state atomically with the exchange; auth codes appear nowhere at rest.
4. **Refresh single-flight:** concurrent `get_access_token()` calls produce one refresh (locking test); failure records `last_refresh_error` + audit and raises `EbayUnavailable`.
5. Connect → status → policy refresh → disconnect all work end-to-end against the **fakes**, emitting the full D-59 audit taxonomy; a test asserts **no audit payload contains a token/code substring**.
6. `/api/ebay/status/` never includes token values; with `EBAY_ENV` unset the app runs normally and status reports "not configured" (D-13 guard).
7. `AuditLog` has no update/delete path (API or admin); the list endpoint filters by action prefix.
8. Frontend: connect/paste/complete flow, env badge, readiness card, disconnect confirm, audit table render against fakes; typecheck + build green.
9. **Zero listing-write code** anywhere (`createOffer` /inventory/media absent — grep-level review check); HTTP confined to `integrations/ebay.py`.
10. Full suites green on the consolidated **"PostgreSQL Validation"** workflow, **plus** the §13 manual sandbox checklist performed once with evidence pasted into the close-out. Production connect **not** required.

10. Test plan

Backend: EncryptedTextField round-trip + ciphertext-at-rest + missing-key error; credential uniqueness per env; OAuthState TTL/single-use/unknown rejection; `complete_connect` happy + each failure path (audited correctly, code never persisted/logged); refresh locking (threaded or sequenced `select_for_update` test) + failure path; disconnect; snapshot upsert + provenance; audit `record()` writes + append-only (no update/delete routes) + secret-free-payload assertion; status endpoint shape incl. unconfigured mode; unauthenticated rejected; backup/restore covers new tables. **Frontend (light):** connect panel flow states; paste-field submit; SANDBOX badge renders from status; readiness card counts; disconnect confirm; `ebay.ts` request shapes. **CI:** consolidated workflow green; assert (review-level) no per-sprint workflow files remain.

---

11. Seed data

No credentials are ever seeded. Seed remains unchanged; the app must boot and the eBay settings page must render sensibly with no credential and `EBAY_ENV` unset. AuditLog starts empty — real events only.

---

12. Out of scope for Sprint 6 (restated — D-64)

Any listing write: media/EPS upload, `createOrReplaceInventoryItem` , `createOffer` , `publishOffer` · post-publish sync / listing-ID storage / item status flips · revise/relist · order/sales sync / `SaleRecord` · Browse or any Buy API (F-03 stands) · Notification API webhooks (F-10 exemption path taken) · public OAuth callback endpoint (paste-back is the baseline; localhost callback optional for sandbox only) · multi-account / multi-channel · KMS/keyring · AI anything.

---

13. Manual sandbox checklist (closure evidence — run once, outside CI)

1. Sandbox keyset configured in env (`EBAY_ENV=sandbox`). 2. Sandbox test user exists. 3. `connect/start` yields a consent URL; consent completed as the test user. 4. Redirected URL pasted; exchange succeeds. 5. DB inspection shows **ciphertext**

token columns. 6. `/api/ebay/status/` shows connected + SANDBOX badge in UI. 7. Token refresh succeeds (force-expire the cached access token). 8. Policy refresh runs (sandbox policy data may be sparse — record what returns). 9. Disconnect clears the credential. 10. Audit log shows the full event sequence. 11. **No token value visible anywhere**: admin, API responses, audit payloads, application logs. — Paste the evidence (screenshots/log lines) into the Sprint 6 close-out message.

Hand-off notes for the implementing agent

- **Tokens are radioactive.** They exist in memory and in two encrypted columns — nowhere else. If a token ever appears in a log line, audit payload, serializer, or test fixture output, that is a defect regardless of passing tests.
- The state nonce is not optional ceremony — validate it exactly as specified.
- `EBAY_ENV` unset must leave every other feature of Magpie untouched (D-13). The fakes are the default, not the fallback.
- If you find yourself writing anything that creates or modifies a listing, an offer, or an image on eBay, you have left the sprint.
- When ambiguous, prefer the smaller implementation that satisfies §9.